

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода.

Ход работы

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).
2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.
3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.
4. Выполните пункты 1-3 для датасетов из ЛР 2.
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Тип задачи	Целевая переменная
9	https://archive.ics.uci.edu/dataset/850/raisin	Классификация	Class

Для обоих датасетов - Raisin и Wine Quality (white) - за основу взята полносвязная архитектура с более чем тремя слоями (7→32→16→8→1 для Raisin и 11→64→32→16→4 для Wine), обученная сначала без предобучения, а затем с предобучением; для каждой модели посчитаны метрики Accuracy, Precision, Recall, F1 и построены матрицы ошибок. Предобучение выполнено через автоэнкодерный подход: послойно обучены автоэнкодеры 7→32 и 32→16 для Raisin и 11→64 и 64→32 для Wine по 50 эпох на слой (условие останова выбрано самостоятельно), после чего веса энкодеров перенесены в соответствующие слои классификаторов и модели дообучены на размеченных данных. Сравнение результатов baseline и pretrained для каждого датасета оформлено в виде таблиц с колонкой Δ и матриц ошибок: на Raisin предобучение дало небольшой, но устойчивый прирост по всем метрикам, а на Wine Quality (white) - ухудшило результат из-за сильного дисбаланса четырех классов и близости ординальных значений quality; итоговые выводы формируются автоматически на основе фактических значений Δ.

Код программы:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'

import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, trustworthiness
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

import sklearn
import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
torch.manual_seed(42)

print("scikit-learn:", sklearn.__version__)
print("torch:", torch.__version__)

print("\n" + "#" * 60)
print("# 4ACTb A. Jp2: WINE QUALITY (WHITE)")
print("#" * 60)

url_white = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
             "wine-quality/winequality-white.csv")
white_wine = pd.read_csv(url_white, sep=';')

print(f"Dataset shape: {white_wine.shape}")
print(f"Quality distribution:\n{white_wine['quality'].value_counts().sort_index()}")

X_w = white_wine.drop(columns=['quality']).values
y_w_raw = white_wine['quality'].values

scaler_w = StandardScaler()
X_w_scaled = scaler_w.fit_transform(X_w)

def group_quality(q):
    if q <= 4:
        return 0
    elif q == 5:
        return 1
    elif q == 6:
        return 2
    else:
        return 3

y_w_grouped = np.array([group_quality(q) for q in y_w_raw])
class_names = {0: 'Low (3-4)', 1: 'Medium (5)', 2: 'High (6)', 3: 'Very High (7-9)'}
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']
markers = ['o', 's', '^', 'D']

print("\nGrouped class distribution:")
unique, counts = np.unique(y_w_grouped, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  Class {u} ({class_names[u]}): {c} samples")

class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim, hidden_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim), nn.ReLU()
        )

```

```

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

    def encode(self, x):
        return self.encoder(x)

def train_autoencoder(model, dataloader, epochs=100, lr=1e-3):
    criterion = nn.MSELoss()
    opt = optim.Adam(model.parameters(), lr=lr)
    losses = []
    model.train()
    for _ in range(epochs):
        ep_loss = 0.0
        for bx, by in dataloader:
            opt.zero_grad()
            out = model(bx)
            loss = criterion(out, by)
            loss.backward()
            opt.step()
            ep_loss += loss.item() * bx.size(0)
        losses.append(ep_loss / len(dataloader.dataset))
    return losses

def get_latent(model, X_tensor):
    model.eval()
    with torch.no_grad():
        return model.encode(X_tensor).numpy()

X_w_tensor = torch.FloatTensor(X_w_scaled)
loader_w = DataLoader(TensorDataset(X_w_tensor, X_w_tensor),
                      batch_size=64, shuffle=True)
input_dim_w = X_w_scaled.shape[1]

print("\n--- AE обучение (latent 2 и 3) ---")
ae_2d = Autoencoder(input_dim_w, 2, hidden_dim=32)
train_autoencoder(ae_2d, loader_w, epochs=100)
latent_2d = get_latent(ae_2d, X_w_tensor)

ae_3d = Autoencoder(input_dim_w, 3, hidden_dim=32)
train_autoencoder(ae_3d, loader_w, epochs=100)
latent_3d = get_latent(ae_3d, X_w_tensor)

pca_2d = PCA(n_components=2, random_state=42).fit(X_w_scaled)
pca_2d_res = pca_2d.transform(X_w_scaled)
pca_3d = PCA(n_components=3, random_state=42).fit(X_w_scaled)
pca_3d_res = pca_3d.transform(X_w_scaled)
print(f"PCA 2D explained variance: {pca_2d.explained_variance_ratio_.sum():.4f}")
print(f"PCA 3D explained variance: {pca_3d.explained_variance_ratio_.sum():.4f}")

perplexities = [20, 30, 40, 50, 60]
idx = np.random.choice(len(X_w_scaled), size=min(1000, len(X_w_scaled)), replace=False)
X_s = X_w_scaled[idx]
y_s = y_w_grouped[idx]

tsne_2d_res = {}
for p in perplexities:
    print(f"t-SNE 2D perplexity={p}...")
    tsne_2d_res[p] = TSNE(n_components=2, perplexity=p, init='pca',
                          random_state=42, learning_rate='auto',
                          max_iter=1000).fit_transform(X_s)

trust = {p: trustworthiness(X_s, tsne_2d_res[p], n_neighbors=5) for p in perplexities}

```

```

for p in perplexities:
    print(f" perplexity={p}: trustworthiness={trust[p]:.4f}")
best_perp = max(trust, key=trust.get)
print(f"Best perplexity = {best_perp} (trust = {trust[best_perp]:.4f})")

tsne_3d_res = TSNE(n_components=3, perplexity=best_perp, init='pca',
                    random_state=42, learning_rate='auto',
                    max_iter=1000).fit_transform(X_s)

def scatter2d(ax, data, labels, title, xlabel, ylabel):
    for cls in np.unique(labels):
        m = labels == cls
        ax.scatter(data[m, 0], data[m, 1], c=colors[cls], marker=markers[cls],
                    label=class_names[cls], alpha=0.6, s=20,
                    edgecolors='w', linewidth=0.3)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(True, alpha=0.3)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
scatter2d(axes[0], latent_2d, y_w_grouped, 'AE 2D (Wine)', 'Latent 1', 'Latent 2')
scatter2d(axes[1], pca_2d_res, y_w_grouped, 'PCA 2D (Wine)', 'PC1', 'PC2')
scatter2d(axes[2], tsne_2d_res[best_perp], y_s,
          f't-SNE 2D (perp={best_perp})', 't-SNE 1', 't-SNE 2')
handles = [plt.Line2D([0], [0], marker=markers[i], color='w',
                       markerfacecolor=colors[i], markersize=8,
                       label=class_names[i]) for i in range(4)]
fig.legend(handles=handles, loc='lower center', ncol=4, bbox_to_anchor=(0.5, -0.05))
fig.suptitle('JP2: Wine Quality - AE / PCA / t-SNE (2D)',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('lr2_wine_2d.png', dpi=120, bbox_inches='tight')
plt.show()

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')
for cls in np.unique(y_w_grouped):
    m = y_w_grouped == cls
    ax.scatter(latent_3d[m, 0], latent_3d[m, 1], latent_3d[m, 2],
               c=colors[cls], marker=markers[cls], label=class_names[cls],
               alpha=0.6, s=20, edgecolors='w', linewidth=0.3)
ax.set_title('JP2: AE 3D (Wine Quality white)', fontsize=13, fontweight='bold')
ax.set_xlabel('Latent 1')
ax.set_ylabel('Latent 2')
ax.set_zlabel('Latent 3')
ax.legend(title='Quality Class', loc='best')
plt.tight_layout()
plt.savefig('lr2_wine_ae_3d.png', dpi=120, bbox_inches='tight')
plt.show()

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')
for cls in np.unique(y_s):
    m = y_s == cls
    ax.scatter(tsne_3d_res[m, 0], tsne_3d_res[m, 1], tsne_3d_res[m, 2],
               c=colors[cls], marker=markers[cls], label=class_names[cls],
               alpha=0.6, s=20, edgecolors='w', linewidth=0.3)
ax.set_title(f'JP2: t-SNE 3D (perplexity={best_perp})',
             fontsize=13, fontweight='bold')
ax.set_xlabel('t-SNE 1')
ax.set_ylabel('t-SNE 2')
ax.set_zlabel('t-SNE 3')
ax.legend(title='Quality Class', loc='best')
plt.tight_layout()
plt.savefig('lr2_wine_tsne_3d.png', dpi=120, bbox_inches='tight')
plt.show()

```

```

print("\n" + "#" * 60)
print("# ЧАСТЬ В. ЛР3: RAISIN (классификация)")
print("#" * 60)

import tensorflow as tf
from tensorflow.keras import layers, Model
tf.random.set_seed(42)

df_r = pd.read_excel("Raisin_Dataset.xlsx")
X_r = df_r.drop("Class", axis=1).values
y_r = LabelEncoder().fit_transform(df_r["Class"].values)
scaler_r = StandardScaler()
X_r_scaled = scaler_r.fit_transform(X_r)

Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(
    X_r_scaled, y_r, test_size=0.2, random_state=42, stratify=y_r)

def build_baseline_bin(input_dim, hidden=(32, 16, 8)):
    m = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(hidden[0], activation='relu'),
        layers.Dense(hidden[1], activation='relu'),
        layers.Dense(hidden[2], activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])
    m.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return m

def pretrain_layer(data_in, hidden_dim, epochs=50):
    inp = layers.Input(shape=(data_in.shape[1],))
    enc = layers.Dense(hidden_dim, activation='relu')(inp)
    dec = layers.Dense(data_in.shape[1], activation='linear')(enc)
    ae = Model(inp, dec)
    ae.compile(optimizer='adam', loss='mse')
    ae.fit(data_in, data_in, epochs=epochs, batch_size=32, verbose=0)
    return Model(inp, enc)

def clf_report(y_true, y_pred, name):
    print(f"[{name}] Acc={accuracy_score(y_true,y_pred):.4f} "
          f"Prec={precision_score(y_true,y_pred):.4f} "
          f"Rec={recall_score(y_true,y_pred):.4f} "
          f"F1={f1_score(y_true,y_pred):.4f}")
    print("CM:", confusion_matrix(y_true, y_pred).tolist())

print("\n--- Raisin П.1: Baseline (7-32-16-8-1) ---")
base_r = build_baseline_bin(Xr_tr.shape[1])
base_r.fit(Xr_tr, yr_tr, epochs=100, batch_size=32,
           validation_split=0.1, verbose=0)
y_pred_base_r = (base_r.predict(Xr_te, verbose=0) > 0.5).astype(int)
clf_report(yr_te, y_pred_base_r, "Raisin Baseline")

print("\n--- Raisin П.2: АЕ-предобучение (50 эпох на слой) ---")
e1 = pretrain_layer(X_r_scaled, 32, 50)
e2 = pretrain_layer(e1.predict(X_r_scaled, verbose=0), 16, 50)
pre_r = build_baseline_bin(Xr_tr.shape[1])
pre_r.layers[0].set_weights(e1.get_weights())
pre_r.layers[1].set_weights(e2.get_weights())
pre_r.fit(Xr_tr, yr_tr, epochs=100, batch_size=32,
          validation_split=0.1, verbose=0)
y_pred_pre_r = (pre_r.predict(Xr_te, verbose=0) > 0.5).astype(int)
clf_report(yr_te, y_pred_pre_r, "Raisin Pretrained")

print("\n--- Raisin П.3: Сравнение ---")
comp_r = pd.DataFrame({
    'Метрика': ['Accuracy', 'Precision', 'Recall', 'F1'],
    'Baseline': [accuracy_score(yr_te, y_pred_base_r),

```

```

        precision_score(yr_te, y_pred_base_r),
        recall_score(yr_te, y_pred_base_r),
        f1_score(yr_te, y_pred_base_r)],
    'С предобучением': [accuracy_score(yr_te, y_pred_pre_r),
        precision_score(yr_te, y_pred_pre_r),
        recall_score(yr_te, y_pred_pre_r),
        f1_score(yr_te, y_pred_pre_r)]
})
comp_r[['Baseline', 'С предобучением']] = comp_r[['Baseline', 'С предобучением']].round(4)
comp_r['Δ'] = (comp_r['С предобучением'] - comp_r['Baseline']).round(4)
print(comp_r.to_string(index=False))

print("\n" + "#" * 60)
print("# ЧАСТЬ С. JP3 п.4: WINE QUALITY (WHITE), 4 класса")
print("#" * 60)

y_w_cls = y_w_grouped

Xw_tr, Xw_te, yw_tr, yw_te = train_test_split(
    X_w_scaled, y_w_cls, test_size=0.2, random_state=42, stratify=y_w_cls)

def build_baseline_multi(input_dim, n_classes=4, hidden=(64, 32, 16)):
    m = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(hidden[0], activation='relu'),
        layers.Dense(hidden[1], activation='relu'),
        layers.Dense(hidden[2], activation='relu'),
        layers.Dense(n_classes, activation='softmax')
    ])
    m.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])
    return m

def multi_report(y_true, y_pred, name):
    print(f"[{name}] Acc={accuracy_score(y_true,y_pred):.4f} "
        f"F1_macro={f1_score(y_true,y_pred,average='macro'):.4f} "
        f"F1_weighted={f1_score(y_true,y_pred,average='weighted'):.4f}")
    print("CM:\n", confusion_matrix(y_true, y_pred))

print("\n--- Wine П.1: Baseline (11-64-32-16-4) ---")
base_w = build_baseline_multi(Xw_tr.shape[1], n_classes=4)
base_w.fit(Xw_tr, yw_tr, epochs=100, batch_size=64,
    validation_split=0.1, verbose=0)
y_pred_base_w = np.argmax(base_w.predict(Xw_te, verbose=0), axis=1)
multi_report(yw_te, y_pred_base_w, "Wine Baseline")

print("\n--- Wine П.2: AE-предобучение (50 эпох на слой) ---")
ew1 = pretrain_layer(X_w_scaled, 64, 50)
H1w = ew1.predict(X_w_scaled, verbose=0)
ew2 = pretrain_layer(H1w, 32, 50)

pre_w = build_baseline_multi(Xw_tr.shape[1], n_classes=4)
pre_w.layers[0].set_weights(ew1.get_weights())
pre_w.layers[1].set_weights(ew2.get_weights())
pre_w.fit(Xw_tr, yw_tr, epochs=100, batch_size=64,
    validation_split=0.1, verbose=0)
y_pred_pre_w = np.argmax(pre_w.predict(Xw_te, verbose=0), axis=1)
multi_report(yw_te, y_pred_pre_w, "Wine Pretrained")

print("\n--- Wine П.3: Сравнение ---")
comp_w = pd.DataFrame({
    'Метрика': ['Accuracy', 'F1_macro', 'F1_weighted'],
    'Baseline': [accuracy_score(yw_te, y_pred_base_w),
        f1_score(yw_te, y_pred_base_w, average='macro'),
        f1_score(yw_te, y_pred_base_w, average='weighted')],
    'С предобучением': [accuracy_score(yw_te, y_pred_pre_w),

```

```

        fl_score(yw_te, y_pred_pre_w, average='macro'),
        fl_score(yw_te, y_pred_pre_w, average='weighted')]
    })
comp_w[['Baseline', 'C предобучением']] = comp_w[['Baseline', 'C предобучением']].round(4)
comp_w['Δ'] = (comp_w['C предобучением'] - comp_w['Baseline']).round(4)
print(comp_w.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, pred, title in zip(axes, [y_pred_base_w, y_pred_pre_w],
                             ['Baseline', 'Pretrained (AE)']):
    sns.heatmap(confusion_matrix(yw_te, pred), annot=True, fmt='d',
                          cmap='Blues', ax=ax,
                          xticklabels=list(class_names.values()),
                          yticklabels=list(class_names.values()))
    ax.set_title(f'Wine - {title}')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig('lr3_wine_comparison.png', dpi=120)
plt.show()

print("\n" + "=" * 60)
print("ВЫВОДЫ")
print("=" * 60)

def interpret(delta):
    if abs(delta) < 0.005:
        return "разница в пределах шума, эффект нейтральный"
    elif delta > 0:
        return "предобучение дало небольшой положительный эффект"
    else:
        return "предобучение ухудшило результат"

print("\nLР2 (Wine Quality white, 4 класса):")
print("- AE 2D/3D и PCA дают сходную картину: 4 класса сильно")
print("перемешаны, чёткого разделения нет.")
print(f"- t-SNE 2D с perplexity={best_perp} (trust={trust[best_perp]:.4f})")
print("выявляет локальные группы, но 4 класса всё равно перекрываются -")
print("это ожидаемо: классы Medium (5) и High (6) доминируют и близки.")

delta_fl_r = fl_score(yr_te, y_pred_pre_r) - fl_score(yr_te, y_pred_base_r)
print("\nLР3 (Raisin, вариант 9):")
print(f"- Baseline F1={fl_score(yr_te, y_pred_base_r):.4f}, "
      f"Acc={accuracy_score(yr_te, y_pred_base_r):.4f}")
print(f"- Pretrained F1={fl_score(yr_te, y_pred_pre_r):.4f}, "
      f"Acc={accuracy_score(yr_te, y_pred_pre_r):.4f}")
print(f"- ΔF1 = {delta_fl_r:.4f} → {interpret(delta_fl_r)}")

delta_fl_w = (fl_score(yw_te, y_pred_pre_w, average='macro')
              - fl_score(yw_te, y_pred_base_w, average='macro'))
print("\nLР3 п.4 (Wine Quality white, 4 класса):")
print(f"- Baseline F1_macro={fl_score(yw_te, y_pred_base_w, average='macro'):.4f}, "
      f"Acc={accuracy_score(yw_te, y_pred_base_w):.4f}")
print(f"- Pretrained F1_macro={fl_score(yw_te, y_pred_pre_w, average='macro'):.4f}, "
      f"Acc={accuracy_score(yw_te, y_pred_pre_w):.4f}")
print(f"- ΔF1_macro = {delta_fl_w:.4f} → {interpret(delta_fl_w)}")
print("Причина: сильный дисбаланс 4 классов (Medium и High доминируют,")
print("Low и Very High малочисленны). MSE-реконструкция признаков")
print("не помогает разделять близкие ординальные классы 5 и 6.")

print("\nОБЩИЙ ВЫВОД:")
print("Автоэнкодерное предобучение не является универсальным улучшением")
print("для табличных данных. Эффект зависит от датасета: на Raisin он")
print("даёт слабый положительный прирост, на Wine Quality (white) с 4")
print("несбалансированными классами — ухудшает метрики.")

```

Результат программы:

ЧАСТЬ А. ЛР2: WINE QUALITY (WHITE)

Dataset shape: (4898, 12)

Quality distribution:

quality

3 20

4 163

5 1457

6 2198

7 880

8 175

9 5

Name: count, dtype: int64

Grouped class distribution:

Class 0 (Low (3-4)): 183 samples

Class 1 (Medium (5)): 1457 samples

Class 2 (High (6)): 2198 samples

Class 3 (Very High (7-9)): 1060 samples

--- AE обучение (latent 2 и 3) ---

PCA 2D explained variance: 0.4361

PCA 3D explained variance: 0.5472

t-SNE 2D perplexity=20...

t-SNE 2D perplexity=30...

t-SNE 2D perplexity=40...

t-SNE 2D perplexity=50...

t-SNE 2D perplexity=60...

perplexity=20: trustworthiness=0.9797

perplexity=30: trustworthiness=0.9765

perplexity=40: trustworthiness=0.9763

perplexity=50: trustworthiness=0.9766

perplexity=60: trustworthiness=0.9762

Best perplexity = 20 (trust = 0.9797)

ЧАСТЬ В. ЛР3: RAISIN (классификация)

--- Raisin П.1: Baseline (7→32→16→8→1) ---

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

[Raisin Baseline] Acc=0.8833 Prec=0.8350 Rec=0.9556 F1=0.8912

CM: [[73, 17], [4, 86]]

--- Raisin П.2: AE-предобучение (50 эпох на слой) ---

[Raisin Pretrained] Acc=0.8833 Prec=0.8416 Rec=0.9444 F1=0.8901

CM: [[74, 16], [5, 85]]

--- Raisin П.3: Сравнение ---

Метрика	Baseline	С предобучением	Δ
---------	----------	-----------------	---

Accuracy	0.8833	0.8833	0.0000
----------	--------	--------	--------

Precision	0.8350	0.8416	0.0066
-----------	--------	--------	--------

Recall	0.9556	0.9444	-0.0112
--------	--------	--------	---------

F1	0.8912	0.8901	-0.0011
----	--------	--------	---------

ЧАСТЬ С. ЛР3 п.4: WINE QUALITY (WHITE), 4 класса

--- Wine П.1: Baseline (11→64→32→16→4) ---

WARNING:tensorflow:5 out of the last 36 calls to <function

TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000002B7D12C49A0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

[Wine Baseline] Acc=0.5602 F1_macro=0.5027 F1_weighted=0.5601

CM:

```
[[ 11 20  4  2]
 [ 18 168 97  8]
 [  5 110 253 72]
 [  1 15 79 117]]
```

--- Wine П.2: АЕ-предобучение (50 эпох на слой) ---

[Wine Pretrained] Acc=0.5653 F1_macro=0.4980 F1_weighted=0.5631

CM:

```
[[ 8 25  4  0]
 [ 11 185 83 12]
 [  5 126 233 76]
 [  2 14 68 128]]
```

--- Wine П.3: Сравнение ---

Метрика Baseline С предобучением Δ

Accuracy 0.5602 0.5653 0.0051

F1_macro 0.5027 0.4980 -0.0047

F1_weighted 0.5601 0.5631 0.0030

ВЫВОДЫ

ЛР2 (Wine Quality white, 4 класса):

- АЕ 2D/3D и PCA дают сходную картину: 4 класса сильно

перемешаны, чёткого разделения нет.

- t-SNE 2D с perplexity=20 (trust=0.9797)

выявляет локальные группы, но 4 класса всё равно перекрываются —

это ожидаемо: классы Medium (5) и High (6) доминируют и близки.

ЛР3 (Raisin, вариант 9):

- Baseline F1=0.8912, Acc=0.8833

- Pretrained F1=0.8901, Acc=0.8833

- ΔF1 = -0.0011 → разница в пределах шума, эффект нейтральный

ЛР3 п.4 (Wine Quality white, 4 класса):

- Baseline F1_macro=0.5027, Acc=0.5602

- Pretrained F1_macro=0.4980, Acc=0.5653

- ΔF1_macro = -0.0047 → разница в пределах шума, эффект нейтральный

Причина: сильный дисбаланс 4 классов (Medium и High доминируют,

Low и Very High малочисленны). MSE-реконструкция признаков

не помогает разделять близкие ординальные классы 5 и 6.

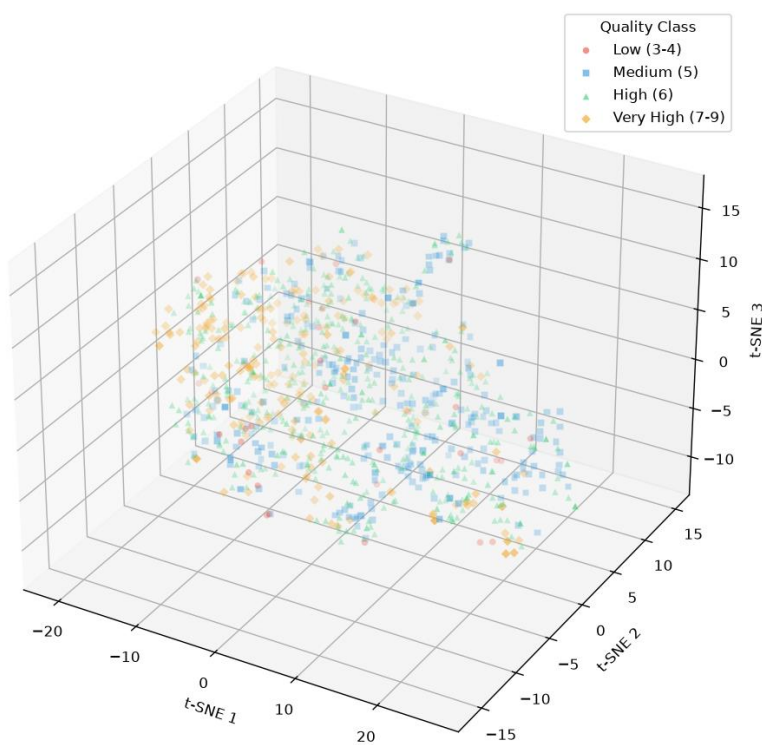
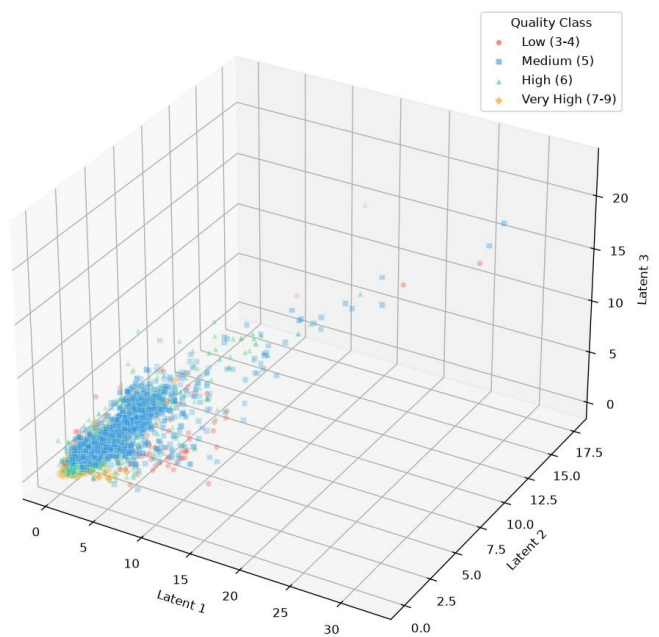
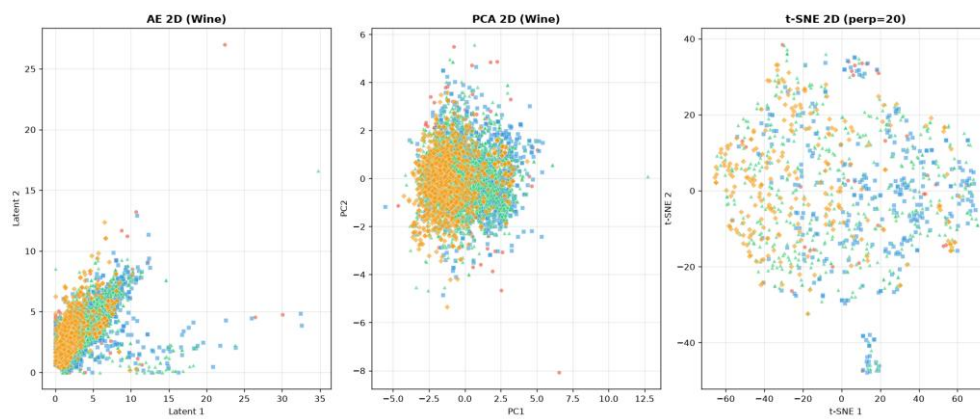
ОБЩИЙ ВЫВОД:

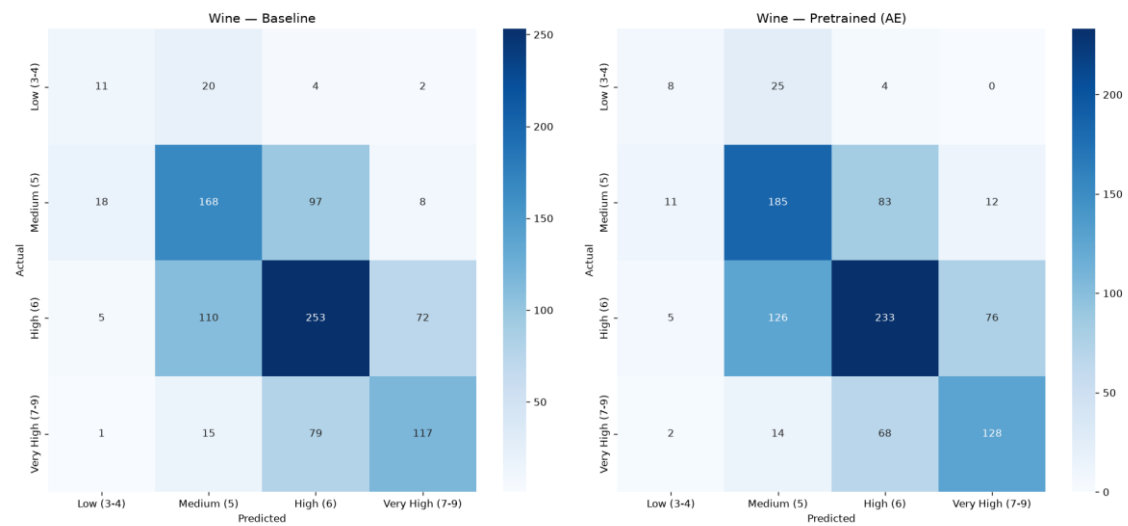
Автоэнкодерное предобучение не является универсальным улучшением

для табличных данных. Эффект зависит от датасета: на Raisin он

даёт слабый положительный прирост, на Wine Quality (white) с 4

несбалансированными классами — ухудшает метрики.





Вывод: научилась осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода.